Module 10 — Slowly Changing Dimensions and Snapshots

Tier: 🟡 Working Effectively · **Duration:** 90 min · **Prerequisites:** Module 09

> ***Why this module exists:*** *Your Silver dimensions store the current state of every contact, deal, and pipeline stage. But stakeholders ask questions about the past: "What stage was this deal in last quarter? What was this contact's email when they signed up?" Without history tracking, you can only answer questions about right now. SCD Type 2 is the standard solution. This module teaches you how dbt native snapshots implement SCD2 — and then explains exactly why we replaced them with a custom macro.*

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Not
00:00	10 min	Recap Module 09	Confirm macro mental model	Q&A	Answer from memory	—	Ask all 4 prep questions. Pr "what does { vs {% %} do in Jinja?" and "v does a macro compile to?" both connect directly to today's snaps config syntax and scd2_me pattern.
00:10	15 min	The SCD2 problem	Understand why overwriting dimension rows destroys analytical value	Present	Annotate example table	This doc	Use the pipe stage scenari Draw before/after the whiteboa Ask: "What's answer to wha stage was dea in on March 1 without SCD2 you cannot answer it.

00:25	15 min	dbt native snapshots	Know snapshot syntax, config options, and both strategies	Present + live demo	Follow along in editor	This doc	Write `snap_contacts` live. Run onc insert data, r again. Show snapshot tab DuckDB.
00:40	10 min	Snapshot metadata columns	Know how to query "as of" a date using snapshot metadata	Present	Write a query	This doc	Show `dbt_valid_fro` `dbt_valid_to`, `dbt_scd_id`. W the point-in-time query together.
00:50	10 min	Why we use scd2_merge instead	Understand the surrogate key stability problem with native snapshots	Present	Listen + discuss	This doc	This is the ke decision. Exp `dbt_scd_id` regeneration full refresh. A "what breaks downstream foreign key changes valu
01:00	5 min	Reading the scd2_merge pattern	Understand what the custom macro generates	Present	Read the macro call	This doc	Walk through the simplified pattern. Don line-by-line i the macro internals — focus on wha goes in and w comes out.
01:05	25 min	Exercise	Build and operate a snapshot end-to-end	Practice	Solo exercise	Exercise below	Circulate. Tas (manual CSV edit) trips pe up — they fo to run `dbt se` before `dbt snapshot`. The bonus query requires a `BETWEEN` or da comparison - watch for off one thinking
01:30	10 min	Debrief + prep	Consolidate	Debrief	Verbal	Whiteboard	Key takeaway "native

		questions					snapshots ar correct — we opted out fo one specific reason: surrogate ke stability." Ask surrogate ke stability wasn concern, wou you use nativ snapshots or scd2_merge? there's no wr answer, explo the tradeoff.

Content

Part A — What SCD Type 2 Is and Why It Matters

The problem: a dimension that overwrites history

Your `dim_contact` table has one row per contact. It stores the current state: email, name, HubSpot contact owner, pipeline stage.

Now imagine contact 42 moves from pipeline stage `lead` to `opportunity` on March 15. Your incremental model runs overnight. The `dim_contact` row is updated:

contact_key	hubspot_contact_id	email	pipeline_stage
abc123	42	anna@example.com	opportunity

The previous row — showing `lead` — is gone. Overwritten. That data no longer exists in your warehouse.

Now a stakeholder asks: **"How many contacts were still in the lead stage at the end of Q1?"**

You cannot answer that question. The warehouse no longer contains that information.

This is the core problem SCD Type 2 solves.

The solution: keep both rows

SCD Type 2 means: when a tracked attribute changes, **close the old row** (set an end date) and **insert a new row** (with a start date). Never overwrite.

After the pipeline stage change, the dimension looks like this:

contact_key	hubspot_contact_id	email	pipeline_stage	valid_from	valid_to
abc123	42	anna@example.com	lead	2024-01-10	2024-03-15
def456	42	anna@example.com	opportunity	2024-03-15	NULL

`NULL` in `valid_to` means "this is the current row." The old row is preserved with its end date.

Now the Q1 query is answerable:

```
SELECT contact_key, pipeline_stage
FROM dim_contact_history
WHERE valid_from  <= '2024-03-31'
  AND (valid_to   >  '2024-03-31' OR valid_to IS NULL)
```

What attributes to track

Not every column should trigger a new SCD2 row. Track only the attributes that:

1. Change over time and
2. Have analytical meaning when they change

For HubSpot contacts: `pipeline_stage`, `contact_owner`, `lifecycle_stage`. Not tracked: `updated_at` itself (it changes on every sync), internal identifiers.

For HubSpot deals: `stage_id`, `close_date`, `deal_owner`.

Part B — dbt Native Snapshots

dbt has a first-class feature for SCD Type 2: **snapshots**. They live in the `snapshots/` folder (not `models/`), use a special `{% snapshot %}` block, and are run with `dbt snapshot` (not `dbt run`).

dbt has a built-in feature called snapshots that automatically handles SCD2 versioning. You define the snapshot once, run `dbt snapshot`, and dbt manages the version rows for you.

Snapshot syntax

```
{% snapshot snap_contacts %}

{{
    config(
        target_schema = 'snapshots',
        unique_key    = 'contact_id',
        strategy      = 'timestamp',
        updated_at    = 'updated_at'
    )
}}

SELECT
    contact_id,
    email,
    pipeline_stage,
    contact_owner,
    updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{% endsnapshot %}
```

The `{% snapshot %}` / `{% endsnapshot %}` block is the snapshot boundary. Everything inside is plain SQL — the `SELECT` is what dbt reads each time `dbt snapshot` runs.

The two snapshot strategies

`timestamp` strategy — uses a datetime column to detect changes.

```
config(
    strategy   = 'timestamp',
    updated_at = 'updated_at'    -- name of the datetime column in your SELECT
)
```

dbt compares `updated_at` in the source against `dbt_updated_at` in the snapshot table. If the source row has a newer timestamp, dbt closes the existing row and inserts a new one.

Requirement: your source must have a reliable `updated_at` column that is always bumped when any tracked column changes. For HubSpot data ingested by Lambda, `updated_at` is present and reliable.

`check` strategy — compares column values directly.

```
config(
    strategy     = 'check',
    check_cols   = ['pipeline_stage', 'contact_owner', 'email']
    -- or check_cols = 'all' to compare every column
)
```

dbt hashes the listed columns on each run. If the hash changes, a new row is inserted.

When to prefer `check`: when your source has no reliable `updated_at`. The trade-off: it requires comparing every tracked column on every row on every run — slower and more compute-intensive for large tables.

	`timestamp`	`check`
Detection method	`updated_at` column comparison	Column value hash comparison
Speed	Fast — single column compare	Slower — hash all check_cols
Requirement	Reliable `updated_at` in source	No timestamp requirement
Risk	Misses changes if `updated_at` not bumped	None — always detects changes

For this project: use `timestamp`. Lambda always bumps `updated_at` on HubSpot records.

What the snapshot table looks like after two runs

After first run (3 contacts in source):

contact_id	email	pipeline_stage	dbt_valid_from	dbt_valid_to	dbt_scd_id
1	anna@example.com	lead	2024-01-10 00:00:00	NULL	`hash_1a`
2	ben@example.com	opportunity	2024-01-10 00:00:00	NULL	`hash_2a`

3	cara@example.com	lead	2024-01-10 00:00:00	NULL	`hash_3a`

After second run (contact 1 has moved to `opportunity`, `updated_at` bumped):

contact_id	email	pipeline_stage	dbt_valid_from	dbt_valid_to	dbt_scd_id
1	anna@example.com	lead	2024-01-10 00:00:00	2024-03-15 09:00:00	`hash_1a`
1	anna@example.com	opportunity	2024-03-15 09:00:00	NULL	`hash_1b`
2	ben@example.com	opportunity	2024-01-10 00:00:00	NULL	`hash_2a`
3	cara@example.com	lead	2024-01-10 00:00:00	NULL	`hash_3a`

The old row for contact 1 was closed (`dbt_valid_to` set). A new row was inserted. Contacts 2 and 3 were unchanged — one row each, `dbt_valid_to` still NULL.

Part C — Snapshot Metadata Columns

Every snapshot table gets four dbt-managed columns added automatically:

Column	Type	What it means
`dbt_valid_from`	timestamp	When this version of the row became active
`dbt_valid_to`	timestamp or NULL	When this version was superseded. NULL = still current
`dbt_scd_id`	varchar	Unique identifier for this specific row version. SHA1 hash of the natural key + valid_from
`dbt_updated_at`	timestamp	When dbt last touched this row (the source's updated_at value at insert time)

Querying "as of" a date

To get the state of every contact as of a specific date (point-in-time query):

```
SELECT
    contact_id,
    email,
    pipeline_stage
FROM snapshots.snap_contacts
WHERE dbt_valid_from  <= '2024-06-01'
  AND (dbt_valid_to    > '2024-06-01' OR dbt_valid_to IS NULL)
```

This returns exactly one row per contact — the version that was active on June 1, 2024.

The NULL check on `dbt_valid_to` is essential: the current row has no end date. Without `OR dbt_valid_to IS NULL`, current rows are excluded from all queries.

The current row only

For reports that only need current state (no history needed), filter to current rows:

```
SELECT contact_id, email, pipeline_stage
FROM snapshots.snap_contacts
WHERE dbt_valid_to IS NULL
```

This is equivalent to querying a standard non-SCD2 dimension.

Part D — Why We Use a Custom scd2_merge Instead of Native Snapshots

> ***Hash terminology — don't confuse these two:***
>
> - *`dbt_scd_id` is a dbt-generated identifier for each snapshot row (SHA1 of unique key + valid_from). It uniquely identifies a specific row version in the snapshot table.*
> - *`row_hash` (in our custom pattern) is an MD5 of the tracked columns — used to detect whether anything changed. If `row_hash` changes between runs, dbt knows a new SCD2 row is needed.*

Native snapshots work correctly. The reason we replaced them is specific: **surrogate key stability**.

The surrogate key stability problem

`dbt_scd_id` is a SHA1 hash of the unique key plus the snapshot's `dbt_valid_from` timestamp. It is the primary key of the snapshot table.

Other models reference snapshot rows via `dbt_scd_id` as a foreign key. For example, `fct_deal` might store the `dbt_scd_id` of the contact's state at deal creation time.

Now suppose you need to do a full refresh of the snapshot — perhaps the snapshot logic changed, or you're recovering from a corrupted run:

```
dbt snapshot --full-refresh
```

dbt drops the snapshot table and rebuilds it from scratch. All rows are re-inserted with new `dbt_valid_from` timestamps — because the timestamps come from when dbt inserts the row, not from the source data. The result: **every `dbt_scd_id` changes**.

Every foreign key in `fct_deal` that pointed to the old `dbt_scd_id` values now points to nothing. Your fact tables have broken foreign keys.

This is the surrogate key stability problem: native snapshot surrogate keys are not stable across full refreshes.

What the custom approach does differently

The custom `scd2_merge` pattern computes the hash key in the **source CTE of the model itself**, not at insert time:

```
WITH source AS (
    SELECT
        contact_id,
```

```
        email,
        pipeline_stage,
        updated_at,
        -- Hash key computed here — stable because it's derived from source data only
        md5(contact_id || pipeline_stage || email) AS contact_scd_key
    FROM {{ ref('stg_hubspot__contacts') }}
)
```

Because the hash is derived from source values (not timestamps), it produces the same result on every run — including full refreshes. A full refresh regenerates the same hash keys. Foreign keys in downstream models remain valid.

This is the one reason we chose the custom approach over native snapshots.

Part E — Understanding the scd2_merge Pattern

The solution to the surrogate key stability problem is to shift hash computation from insert-time (where dbt controls it and regenerates it on full-refresh) to source-time (where you compute it in the model's source CTE, stable across any rebuild).

The custom `scd2_merge` macro is a macro that generates SCD2 merge SQL. You call it at the bottom of a Silver dimension model.

A Silver model using the pattern looks like this:

```
{{ config(
    materialized  = 'incremental',
    unique_key    = 'contact_scd_key',
    on_schema_change = 'sync_all_columns'
) }}

WITH source AS (
    SELECT
        contact_id,
        email,
        pipeline_stage,
        contact_owner,
        updated_at,
        -- Surrogate key: hash of natural key + tracked attributes
        md5(contact_id || '|' || pipeline_stage || '|' || contact_owner) AS contact_scd_key,
        -- Row hash: detects any change in tracked columns
        md5(email || '|' || pipeline_stage || '|' || contact_owner)      AS row_hash
    FROM {{ ref('stg_hubspot__contacts') }}
),

{% if is_incremental() %}

-- On incremental runs: compare incoming rows against current dimension state
current_state AS (
    SELECT contact_id, row_hash, valid_to
    FROM {{ this }}
    WHERE valid_to IS NULL   -- only current rows
```

```
),

changed AS (
    SELECT source.*
    FROM source
    LEFT JOIN current_state USING (contact_id)
    WHERE current_state.contact_id IS NULL          -- new contact
       OR source.row_hash != current_state.row_hash  -- tracked attribute changed
)

SELECT
    contact_scd_key,
    contact_id,
    email,
    pipeline_stage,
    contact_owner,
    updated_at          AS valid_from,
    CAST(NULL AS DATE)  AS valid_to,
    row_hash
FROM changed

{% else %}

-- On first run / full refresh: load all rows
SELECT
    contact_scd_key,
    contact_id,
    email,
    pipeline_stage,
    contact_owner,
    updated_at         AS valid_from,
    CAST(NULL AS DATE) AS valid_to,
    row_hash
FROM source

{% endif %}
```

What this achieves:

1. `contact_scd_key` is computed from source data → stable across full refreshes.
2. `row_hash` detects changes in tracked columns — you control which columns are included.
3. On incremental runs, only changed rows are processed.
4. On full refresh (`dbt run --full-refresh`), all rows are reloaded — but surrogate keys are identical because they're derived from the same source values.

The actual `scd2_merge` macro wraps this pattern into a reusable call. The key insight is **where the hash is computed**: in the source CTE, from data values, not from insert timestamps.

Live Demo Script

This demo takes approximately 10–12 minutes. Run it during Part B.

Step 1 — Create the snapshot file

Create `snapshots/snap_contacts.sql` in the exercise project:

```
{% snapshot snap_contacts %}

{{
    config(
        target_schema = 'snapshots',
        unique_key    = 'contact_id',
        strategy      = 'timestamp',
        updated_at    = 'updated_at'
    )
}}

SELECT
    contact_id,
    first_name,
    last_name,
    email,
    pipeline_stage,
    updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{% endsnapshot %}
```

Step 2 — Run the snapshot (initial load)

```
dbt snapshot
```

Show the output: `Created snapshot snapshots.snap_contacts`. Open DuckDB and run:

```
SELECT * FROM snapshots.snap_contacts ORDER BY contact_id;
```

Point out: every row has `dbt_valid_to = NULL` (all current), and `dbt_valid_from` is the same timestamp for all rows (the moment you ran `dbt snapshot`).

Step 3 — Simulate a change

Open `seeds/raw_contacts.csv`. Change one contact's `email` (or `pipeline_stage`) and bump their `updated_at` by one day. Run:

```
dbt seed
dbt snapshot
```

Step 4 — Show the result

```sql
SELECT contact_id, email, pipeline_stage, dbt_valid_from, dbt_valid_to
FROM snapshots.snap_contacts
ORDER BY contact_id, dbt_valid_from;
```

Show: the modified contact now has **two rows** — one closed (with a `dbt_valid_to`) and one current (with `dbt_valid_to = NULL`). All other contacts still have one row.

Ask: "If you ran `dbt snapshot --full-refresh` right now, what would happen to the `dbt_scd_id` values?"

Exercise (25 min)

> ***Project context:*** *`stg_hubspot__contacts` is already built. You are adding snapshot tracking for contacts — the first SCD2 layer in the exercise project.*

Task 1 — Create `snapshots/snap_contacts.sql`

Create the file `snapshots/snap_contacts.sql` in the exercise project. Use the `timestamp` strategy with `updated_at` as the source column. Track these columns: `contact_id`, `first_name`, `last_name`, `email`, `pipeline_stage`, `updated_at`.

Set `target_schema = 'snapshots'` and `unique_key = 'contact_id'`.

▶ Expected solution

Task 2 — Build the initial snapshot

Run `dbt snapshot` to create the snapshot table. Verify it was created by querying:

```sql
SELECT COUNT(*) FROM snapshots.snap_contacts;
```

You should see the same number of rows as in `raw_contacts`. All rows should have `dbt_valid_to = NULL`.

Task 3 — Simulate a change and observe SCD2 in action

These steps must be run in order. Skipping step 2 is the most common mistake.

- **Step 1: Edit `seeds/raw_contacts.csv`** — Find `contact_id = 3`. Change their `email` to any new address. Bump their `updated_at` by one day (e.g., if it was `2024-05-10 08:00:00`, change it to `2024-05-11 08:00:00`).
- **Step 2: Run `dbt seed`** — this reloads the changed CSV data into DuckDB (if you skip this, the snapshot will see no changes).
- **Step 3: Run `dbt snapshot`** — dbt now detects the change and inserts a new SCD2 row.
- **Step 4: Query the snap_contacts table and find two rows for the changed contact:**

```sql
SELECT
    contact_id,
    email,
    dbt_valid_from,
    dbt_valid_to
FROM snapshots.snap_contacts
```

```
WHERE contact_id = 3
ORDER BY dbt_valid_from;
```

You should see exactly **two rows** for contact 3: the original email with a `dbt_valid_to` date, and the new email with `dbt_valid_to = NULL`.

Bonus — Answer a historical question

Write a SQL query that answers: **"What email address did contact_id 3 have on 2024-06-01?"**

Use only the snapshot table. Do not look at `raw_contacts`.

▶ Expected solution

Reference Material

- dbt snapshots documentation
- dbt snapshot strategies
- Kimball SCD Type 2 reference
- Internal: `dimensional-modeling` skill — covers SCD decision framework (Type 1 vs Type 2 vs Type 3)
- Internal: `dbt-standards` skill — covers when scd2_merge is required vs. native snapshots acceptable

---

Prep Questions for Module 11

1. What does `dbt_valid_to = NULL` mean in a snapshot table?
2. What is the difference between the `timestamp` and `check` snapshot strategies, and when would you choose `check`?
3. Why does running `dbt snapshot --full-refresh` break downstream foreign key references when using native snapshots?
4. In the custom scd2_merge pattern, where is the surrogate key hash computed, and why does that location matter?